

Track-System – Backend Files Map

Last updated: 10 Oct 2026 | Covers everything up to step 4 (login, logout, logs, JWT) | What each file is for and how they are wired together | **[NEW]** = added in step 4

1. Where everything sits

```
Track-System/
  Frontend/ (React + Vite - already built)
    src/pages/Login, ForgotPassword, ChangePassword
  Backend/
    TrackSystem.Api/
      TrackSystem.Api.csproj <- list of libraries + UserSecretsId
      appsettings.json       <- settings (connection string, Jwt section)
      Program.cs             <- the server: setup + endpoints
    Data/
      AppDbContext.cs        <- the bridge to the database
    Models/
      Employee.cs            <- C# classes that mirror tables
      LoginLog.cs            <- one row of dbo.Employees
      LoginLog.cs            <- [NEW] one row of dbo.LoginLogs
    Dtos/
      LoginRequest.cs        <- [NEW] shapes of data sent by the browser
      login.http              <- [NEW] the login body (eNum + password)
    login.http               <- [NEW] local test requests (ignored by git)
    Docs/
      Backend-Files-Map.pdf  <- this file
      Backend-Glossary.pdf   <- the words to learn
    bin/ obj/ (build output - ignored by git)
    .gitignore (what git must not upload)

Outside the repo, on each PC: [NEW]
C:\Users\<you>\AppData\Roaming\Microsoft\UserSecrets\<id>\secrets.json
  <- the JWT signing key (Jwt:Key)
```

2. What each file does

File	What it is	What it does	When do I touch it
TrackSystem.Api.csproj	Project definition	Lists the .NET version and every library. [NEW] Now also BCrypt.Net-Next (password hashing) and Microsoft.AspNetCore.Authentication.JwtBearer (tokens), plus the UserSecretsId: the name of this project's secrets folder (not a secret itself).	Every time we add a new library
appsettings.json	Settings file (not code)	Holds the connection string. [NEW] Also a Jwt section with the public token settings: Issuer, Audience and ExpiresMinutes (60). The signing key is NOT here.	When the server name, the DB or a token setting changes
[NEW] secrets.json (User Secrets)	Secret settings, outside the repo	Holds Jwt:Key, the key that signs and checks every token. Each PC has its own. .NET merges it with appsettings.json at startup, so the code reads both the same way.	Only with the dotnet user-secrets commands
Data/AppDbContext.cs	The bridge to the DB	Represents the database in C#. One DbSet line per table. [NEW] Now holds Employees and LoginLogs.	Every time we add a table
Models/Employee.cs	A model (a shape)	One row of dbo.Employees. The Password property now holds a BCrypt hash, never the real password.	When a column in dbo.Employees changes
[NEW] Models/LoginLog.cs	A model (a shape)	One row of dbo.LoginLogs: one login attempt (logID, eNum, loginDate, ipAddress, success).	When a column in dbo.LoginLogs changes
[NEW] Dtos/LoginRequest.cs	A DTO (input shape)	Describes exactly what the browser sends to the login: eNum and password. ASP.NET turns the JSON body into this object automatically.	If the login form sends other fields
Program.cs	The server	Setup area, then endpoints. [NEW] The setup now reads the JWT settings (stops with a clear message if Jwt:Key is missing), registers the token guard and the rules system, and puts them in the request pipeline. New endpoint: POST /api/auth/login.	Every new endpoint or new service

File	What it is	What it does	When do I touch it
.gitignore	Git rules	Stops build files and secret files from being uploaded. [NEW] Also ignores *.http test files.	Rarely
[NEW] login.http	Local test file (REST Client)	Ready-made requests for the login and the protected endpoint. Not in git, because it holds a test password.	When testing endpoints by hand

3. How the files are connected

```

Browser / React --(HTTP request /api/... + "Authorization: Bearer <token>")--> Program.cs
|
| [NEW] UseAuthentication: checks the token with the key from ---> secrets.json
|                       and Issuer / Audience from -----> appsettings.json
| [NEW] UseAuthorization: blocks protected endpoints without a valid token (401)
|
| Program.cs reads the connection string from --> appsettings.json
| the login body arrives as -----> Dtos/LoginRequest.cs [NEW]
| Program.cs uses the classes from -----> Data/AppDbContext.cs
| AppDbContext knows each table's shape from ----> Models/Employee.cs, Models/LoginLog.cs [NEW]
| all of it is possible only because -----> .csproj installed EF Core, BCrypt, JwtBearer
v
AppDbContext --(EF Core turns C# into SQL)--> SQL Server, DB FinalProject

```

This file	needs / uses	Why
Program.cs	appsettings.json	Reads "DefaultConnection". [NEW] Also Jwt:Issuer, Jwt:Audience and Jwt:ExpiresMinutes.
[NEW] Program.cs	secrets.json (User Secrets)	Reads Jwt:Key. The same key signs tokens at login and checks them on every request. Without it the server refuses to start.
[NEW] Program.cs (/api/auth/login)	Dtos/LoginRequest.cs	The login endpoint receives the body as a LoginRequest object.
[NEW] Program.cs (/api/auth/login)	AppDbContext.Employees	Finds the employee, checks the hash with BCrypt, updates failedLoginAttempts / lockoutEnd.
[NEW] Program.cs (/api/auth/login)	AppDbContext.LoginLogs	Adds one row per attempt. The row and the employee changes are saved together with SaveChangesAsync.
Program.cs (/api/employees)	AppDbContext.Employees	Reads the employees and picks only the safe fields. [NEW] Runs only if the request carries a valid token.
AppDbContext.cs	Models/Employee.cs, [NEW] Models/LoginLog.cs	Each DbSet line says which model shapes which table.
[NEW] Models/LoginLog.cs	dbo.LoginLogs (the real table)	Property names must match the columns. eNum has no foreign key by design (unknown numbers are logged too).
Everything	TrackSystem.Api.csproj	If a library version does not match the .NET version, nothing builds.

4. The addresses that exist today (all inside Program.cs)

Address	Needs a token?	Answers	What it proves
GET /api/health	No	{"status":"ok"}	The server itself is alive
GET /api/db-check	No	{"database":"connected"}	The server reached the FinalProject database
[NEW] POST /api/auth/login	No (this is where you get one)	200: message, eNum, mustChangePassword, token. 401: "Invalid employee number or password". 423: "Account is locked. Try again in 15 minutes."	BCrypt check, 5-try lockout, logging of every attempt, token issued
GET /api/employees	[NEW] Yes	With a valid token: the list (no passwords). Without, or with a fake, edited or expired token: 401.	The token guard works

5. Rules for this project

Rule	Why
Services are registered above <code>var app = builder.Build();</code>	After the server is built, nothing can be registered any more.
[NEW] UseAuthentication, then UseAuthorization, right after Build() and before the endpoints	Program.cs builds the request pipeline top to bottom: identify the caller first, then apply the rules.
Server and DB names live in appsettings.json. Secret keys go to User Secrets.	One place to change, and nothing secret reaches the public GitHub repo.
All database traffic goes through AppDbContext	One single door to the DB.
The model may hold the password hash; an endpoint never sends it	Each endpoint chooses which fields leave the server.
[NEW] Never put secrets in a token	The token payload is readable by anyone. Only IDs and flags go in it.
[NEW] Times used by code are UTC; times read by people are Israel time	lockoutEnd is UTC (no daylight-saving jumps). LoginLogs.loginDate is Israel time. Sort logs by logID.
[NEW] A DB structure change is run on both machines and written in the Drive docs	The DB is not in git; each PC has its own copy.
After a code or appsettings change: save, stop (Ctrl+C), run again	The server reads both only at startup.
Run dotnet commands inside Backend\TrackSystem.Api	From the repo root they fail with MSB1003.
bin/, obj/, secrets and *.http files stay out of git	Run git status before every commit.

6. Open items

Item	Details
[NEW] mustChangePassword not enforced yet	A first-login token currently opens every protected endpoint. With the ChangePassword step, such a token will open only the change-password endpoint.
[NEW] /api/employees: any logged-in user	It needs a token, but any role may call it. Later: only a manager or a user with permission (role rules).
[NEW] Rate limiting (planned piece 5.4)	Limit login attempts per IP (e.g. 10 per minute, answer 429), against bots trying many numbers.
[NEW] Equal response time (planned piece 5.5)	Unknown eNums skip BCrypt and answer faster; a dummy check will hide which numbers exist.
[NEW] Employee status	Status 'pending' is not checked at login yet. Waiting for the spec decision.
[NEW] CORS + React	Allow the React port, connect the Login page with axios, send the token, 401 -> back to Login.
[NEW] DB changes not tracked in git	Planned Database/changes/ folder with .sql files (e.g. dropping the LoginLogs FK).
Shared server name	If a PC's SQL instance has another name, override DefaultConnection in User Secrets on that PC only, instead of editing appsettings.json.
/api/db-check is open	A development tool; remove or protect it before any real deployment.
Files not created yet	Models for the other 8 tables, endpoint files (to keep Program.cs short), output DTOs, a Services layer for the login logic, and an xUnit test project.